Адаптация LLM с интеграцией коллаборативной семантики

Адаптация больших языковых моделей с интеграцией коллаборативной семантики для рекомендаций

---

1. Проблематика и контекст

1.1 Семантический разрыв

Рекомендательные системы на основе коллаборативной фильтрации (CF) и большие языковые модели (LLM) решают, по сути, похожие задачи — понять предпочтения пользователя и предложить ему что-то нужное. Проблема в том, что они делают это в совершенно разных репрезентационных пространствах, между которыми нет прямого перехода.

В CF предметы и пользователи представлены плотными вещественными векторами, обученными на графе взаимодействий. Два предмета оказываются близко в этом пространстве, если пользователи, кликнувшие на один, часто кликали и на другой. Классическая формулировка через BPR:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)\in\mathcal{D}} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj})$$

где i — наблюдаемый положительный предмет, j — отрицательный пример. На выходе получаем матрицу эмбеддингов $\mathbf{E} \in \mathbb{R}^{|\mathcal{I}|\times d_{\text{cf}}}$, которая несёт исключительно поведенческую семантику — текстовое содержание предмета в ней никак не отражено.

LLM работают принципиально иначе: трансформеры принимают на вход дискретные последовательности токенов из словаря $\mathcal{V}$ размером 30K–100K. При предобучении на текстах интернета модель никогда не видела числовых ID предметов, поэтому `[ITEM_42]` для неё — просто бессмысленная строка без каких-либо ассоциаций.

Разрыв между этими двумя системами можно коротко описать так:

Свойство	CF-эмбеддинги	Пространство токенов LLM
Представление	Плотный вещественный вектор	Дискретный ID токена
Что кодирует	Поведенческое сосуществование	Лингвистическое значение
Размерность	$d_{\text{cf}} \approx 64$–256	$d_{\text{lm}} \approx 768$–$4096$
Откуда берётся	Граф взаимодействий	Текстовые корпусы

1.2 Почему нельзя просто взять и подать ID в LLM

Классические последовательные рекомендательные системы — SASRec, BERT4Rec — работают с последовательностями ID:

$$[i_1, i_2, \ldots, i_{n-1}] \xrightarrow{\text{Transformer}} \hat{i}_n$$

Если попытаться встроить сюда предобученный LLM, возникает сразу несколько несовместимостей. ID предметов (числа от 1 до сотен тысяч) не существуют как семантические единицы в словаре LLM, поэтому добавить их туда — значит инициализировать эмбеддинги случайно, без какого-либо prior. Файн-тюнинг полной модели на этих ID разрушает языковые представления, накопленные за предобучение (catastrophic forgetting). И наконец, для новых предметов CF-эмбеддингов нет вовсе, а LLM без текстовых данных не может компенсировать этот пробел.

1.3 Что предлагалось до этой статьи и почему не работало

Наиболее прямолинейный подход — перевести всё в текст. P5 и TALLRec заменяют ID предметов их текстовыми описаниями и файн-тюнят T5 или LLaMA на рекомендационных промптах. Это работает на cold-start, но полностью теряет поведенческий сигнал: два предмета с одинаковым текстом, но разными паттернами взаимодействий для такой модели неразличимы.

Другой вариант — добавить ID предметов как специальные токены и файн-тюнить LLM целиком. Вычислительно это неприемлемо при миллиардах параметров, плюс снова catastrophic forgetting.

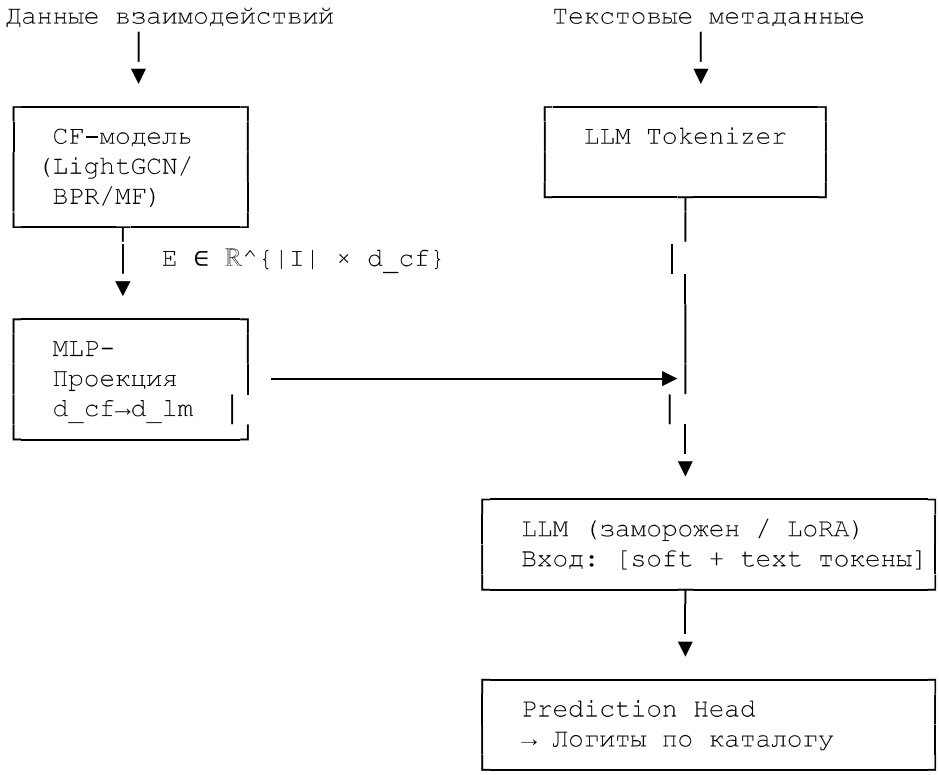
Soft prompt tuning кажется компромиссом: вместо дискретных токенов — обучаемые непрерывные векторы. Но они инициализируются случайно и не опираются ни на какой внешний сигнал, то есть по сути это обучение с нуля поверх замороженного LLM.

Общая проблема всех этих подходов — ни один не использует предобученные CF-эмбеддинги напрямую как входной сигнал для LLM. Именно эту идею реализует данная статья.

---

2. Архитектура метода

2.1 Общая схема



Идея проста: CF-модель предобучается отдельно на графе взаимодействий и даёт нам матрицу эмбеддингов $\mathbf{E}_{\text{cf}}$. Затем небольшой MLP обучается переводить эти векторы в пространство LLM, чтобы их можно было вставлять в последовательность наравне с обычными токенами. Сам LLM при этом либо полностью заморожен, либо адаптируется через LoRA.

2.2 Предобучение CF-модели

Обучаем CF на матрице взаимодействий $\mathbf{R} \in \{0,1\}^{|\mathcal{U}|\times|\mathcal{I}|}$. Авторы предпочитают LightGCN, где распространение по графу задаётся как:

$$\mathbf{E}^{(l+1)} = \tilde{\mathbf{A}}\,\mathbf{E}^{(l)}, \quad \mathbf{E}_{\text{cf}} = \frac{1}{L+1}\sum_{l=0}^{L}\mathbf{E}^{(l)}$$

$\tilde{\mathbf{A}}$ — симметрично-нормированная матрица смежности двудольного графа пользователь–предмет. После этого шага $\mathbf{E}_{\text{cf}} \in \mathbb{R}^{|\mathcal{I}|\times d_{\text{cf}}}$ замораживается и больше не обновляется.

2.3 Проекционный слой

Обучаемый MLP переводит CF-эмбеддинг в пространство LLM:

$$\mathbf{h}_1 = \text{GELU}(\mathbf{W}_1\mathbf{e}_i^{\text{cf}} + \mathbf{b}_1)$$

$$\mathbf{p}_i = \text{LayerNorm}(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$

Размерности по шагам:

Переменная	Форма	Описание
$\mathbf{e}_i^{\text{cf}}$	$(d_{\text{cf}},)$	CF-эмбеддинг предмета i
$\mathbf{W}_1$	$(d_{\text{hidden}},\ d_{\text{cf}})$	Первый линейный слой
$\mathbf{h}_1$	$(d_{\text{hidden}},)$	Скрытая активация
$\mathbf{W}_2$	$(d_{\text{lm}},\ d_{\text{hidden}})$	Второй линейный слой
$\mathbf{p}_i$	$(d_{\text{lm}},)$	Soft-токен для LLM

Типичные значения: $d_{\text{cf}} = 64$, $d_{\text{hidden}} = 256$, $d_{\text{lm}} = 768$.

2.4 Конструирование входной последовательности

Для истории пользователя $\mathcal{S}_u = [i_1, i_2, \ldots, i_n]$ строим входную последовательность как чередование soft-токенов и текстовых токенов:

```
[INST] История покупок:
  <cf_{i1}> "Название предмета 1",
  <cf_{i2}> "Название предмета 2",
  ...
Предскажи следующий предмет. [/INST]
```

Каждый `<cf_{ik}>` — это не строка, а вектор $\mathbf{p}_{i_k} \in \mathbb{R}^{d_{\text{lm}}}$, вставленный в выход embedding-слоя до того, как последовательность попадает в трансформер:

$$\mathbf{X} = [\mathbf{x}_{\text{sys}},\ \mathbf{p}_{i_1},\ \mathbf{x}_{t_1},\ \mathbf{p}_{i_2},\ \mathbf{x}_{t_2},\ \ldots,\ \mathbf{p}_{i_{n-1}},\ \mathbf{x}_{t_{n-1}}] \in \mathbb{R}^{T\times d_{\text{lm}}}$$

2.5 Forward Pass через LLM

Объединённая последовательность $\mathbf{X}$ проходит через LLM:

$$\mathbf{H} = \text{LLM}(\mathbf{X}) \in \mathbb{R}^{T\times d_{\text{lm}}}$$

Представление на последней позиции используется для предсказания следующего предмета:

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{out}}\,\mathbf{H}[-1,:] \in \mathbb{R}^{|\mathcal{I}|}$$

$$\hat{p}(i \mid \mathcal{S}_u) = \mathrm{softmax}(\hat{\mathbf{y}})_i$$

2.6 Функции потерь

Основная потеря — кросс-энтропия на предсказание следующего предмета:

$$\mathcal{L}_{\text{rec}} = -\frac{1}{|\mathcal{D}|} \sum_{(\mathcal{S}_u,\, i^+) \in \mathcal{D}} \log \hat{p}(i^+ \mid \mathcal{S}_u)$$

Вторая потеря — структурная. Смысл в том, что MLP не должен ломать геометрию CF-пространства: если два предмета были похожи по поведению, они должны остаться похожими после проекции:

$$\mathcal{L}_{\text{align}} = \frac{1}{|\mathcal{B}|^2} \sum_{i,j \in \mathcal{B}} \Big(\cos(\mathbf{p}_i, \mathbf{p}_j) - \cos(\mathbf{e}_i^{\text{cf}}, \mathbf{e}_j^{\text{cf}}) \Big)^2$$

Опционально добавляется языковая потеря для сохранения лингвистических способностей LLM:

$$\mathcal{L}_{\text{lm}} = -\sum_t \log p_{\text{LLM}}(w_t \mid w_{<t},\, \mathbf{X})$$

Итоговый обучающий сигнал:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \alpha \cdot \mathcal{L}_{\text{align}} + \beta \cdot \mathcal{L}_{\text{lm}}$$

где $\alpha \in [0.1, 1.0]$, $\beta \in [0.01, 0.1]$ — подбираются через ablation.

---

3. Обучение и оценка

3.1 Датасеты

Авторы тестируют метод на трёх типах данных.

MovieLens-1M / 20M — оценки фильмов (~6K и ~27K фильмов соответственно). Стандартный бенчмарк для последовательной рекомендации, позволяет напрямую сравниться с SASRec и BERT4Rec.

Amazon Product Reviews (Beauty, Sports, Electronics) — отзывы на товары e-commerce, 200K–1.2M взаимодействий. Данные разреженные, применяется 5-core filtering (не менее 5 взаимодействий у пользователя и предмета). Хорошо тестирует работу в условиях cold-start.

Yelp — ~1.6M отзывов на заведения. Нетоварный домен, проверяет, насколько метод обобщается за пределы e-commerce.

Везде используется протокол leave-one-out: последнее взаимодействие — тест, предпоследнее — валидация, всё остальное — обучение.

3.2 Метрики

Два стандарта для оценки ранжирования: HR@K и NDCG@K.

$$\text{HR@K} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbf{1}[i_u^* \in \text{Top-K}(\hat{\mathbf{y}}_u)]$$

HR@K — бинарная: был ли правильный предмет в топе? Не различает позицию 1 и позицию K.

$$\text{NDCG@K} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\mathbf{1}[i_u^* \in \text{Top-K}(\hat{\mathbf{y}}_u)]}{\log_2(\text{rank}(i_u^*) + 1)}$$

NDCG@K штрафует за то, что правильный ответ оказался ниже в списке — это важно, потому что пользователи редко просматривают весь топ. Обе метрики нужны вместе: высокий HR при низком NDCG означает, что модель угадывает, но ставит правильный предмет в конец списка.

3.3 Сравниваемые модели

Модель	Тип
BPR-MF, LightGCN	Классические CF
SASRec, BERT4Rec	Последовательные CF
P5	LLM только на тексте
TALLRec	LLM с промптами
LC-Rec	LLM + CF (предлагаемый метод)

---

4. Критический анализ

4.1 Задержка

Полный инференс включает lookup CF-эмбеддингов ($O(1)$), MLP-проекцию ($O(d_{\text{cf}} \cdot d_{\text{lm}})$) и forward pass через LLM — $O(T^2 \cdot d_{\text{lm}})$ на каждый из L слоёв. Для LLaMA-7B с $d_{\text{lm}} = 4096$, $L = 32$, $T = 512$ это порядка 200–500 мс на запрос на A100.

Реальный SLA у крупных платформ — меньше 50 мс P99. Метод в 10–100 раз медленнее без специальной инженерии: кэширования скрытых состояний, дистилляции в компактную модель или ANN-поиска вместо полного прохода.

4.2 Масштабируемость

Output head $\mathbf{W}_{\text{out}} \in \mathbb{R}^{|\mathcal{I}| \times d_{\text{lm}}}$ растёт линейно с каталогом. При $|\mathcal{I}| = 350M$ (Amazon) и $d_{\text{lm}} = 768$ это около 1 ТБ только для весов головы — не говоря об инференсе. Очевидное решение — ANN-поиск поверх эмбеддингов, но это требует отдельной архитектурной доработки, которой в статье нет.

4.3 Стоимость обучения

Один эпох на MovieLens-1M с LLaMA-7B обходится в $500–2000 на облачных GPU. SASRec с той же задачей обучается за минуты на обычном железе. Разница в 100–1000 раз — это не абстрактное число, это реальный барьер для воспроизведения и адаптации метода в небольших командах.

4.4 Cold-start

Для новых предметов, у которых ещё нет CF-эмбеддинга, MLP просто нечего обрабатывать. В статье не предложено никакого fallback-механизма — авторы признают это ограничением, но не решают его. Новые пользователи без истории взаимодействий тоже ставят модель в тупик: последовательность пустая, и она фактически деградирует до рекомендации по популярности.

4.5 Риск утечки данных

Здесь есть два момента, которые авторы недостаточно явно обсуждают. Во-первых, если CF-эмбеддинги обучены на полном датасете без разбивки по времени, модель косвенно «видит» тестовые взаимодействия через CF-prior ещё до начала обучения LLM-части. Во-вторых, LLM предобучены на

интернете, где есть рецензии на фильмы и товары — часть улучшения метрик может объясняться меморизацией, а не рекомендационным обучением как таковым.

4.6 Валидность экспериментального дизайна

SASRec в статье сравнивается без тщательного подбора гиперпараметров, тогда как для LC-Rec используется явная оптимизация. При должной настройке SASRec нередко догоняет LLM-методы на плотных датасетах.

Кроме того, авторы используют sampled evaluation — HR и NDCG считаются по топ-K среди 100 случайных негативов, а не по всему каталогу. Это стандартная практика, но она завышает метрики для popularity-biased моделей и делает результаты несопоставимыми с работами, использующими full-ranking evaluation.

---

5. Заключение

Метод предлагает элегантное решение реальной проблемы: MLP-проекция позволяет передать поведенческий сигнал CF напрямую в LLM без переобучения последнего с нуля. Это архитектурно аккуратно и работает.

При этом применимость в production остаётся ограниченной. Задержка и стоимость обучения делают метод неподходящим для систем с жёсткими SLA и большими каталогами. Там, где данные плотные и поведенческий сигнал хорошо покрывает каталог, SASRec при правильной настройке даст сопоставимый результат за долю вычислительных затрат.

Наиболее интересные сценарии для применения — задачи с частым cold-start, кросс-доменные рекомендации и диалоговые системы, где языковые возможности LLM нужны сами по себе. Статья закладывает принципиальный фундамент, но до production-масштаба методу ещё нужно пройти через дистилляцию, квантизацию и нормальную интеграцию с ANN-поиском.